

AIQT Milestone 9 Build Specification v0.3

Table of Contents

AIQT Milestone 9 Build Specification v0.3

Milestone: M9

Title: Workflow Manager, Review Acknowledgment, and Release Readiness Exports

Status: Build-ready draft

Depends on: M1-M8, including M8 Agent Driver Protocol and Stdin Input

Primary dogfood evidence: Natural Medicine Marketplace completed 26/26 work units, reached projectStatus = review, but aiqt review remained failed because of one historical partial checkpoint finding from WU003.

1. Source Alignment

M9 is a corrective milestone informed by the completed Natural Medicine Marketplace dogfood.

The dogfood validated the core AIQT loop:

state -> graph -> packet -> implementation -> checkpoint -> dependency recalculation -> next packet -> milestone transition -> review state

The core execution engine should not be redesigned in M9. The defects exposed by dogfood were concentrated in the management, terminal-review, recovery, and export layers.

M9 must preserve the existing AIQT product principles:

- AIQT is a local state engine, not an AI runner.
- .aiqt/project.json and .aiqt/state.json remain the source of truth.
- .aiqt/runlog.jsonl remains append-only workflow history.
- Exports remain generated views, not canonical source files.
- Agent packets remain bounded to one work unit.
- Human-readable documents are generated when useful, not the primary state model.

Dogfood terminal state to support

The final dogfood state was:

projectStatus = review

work units done = 26

ready = 0
planned = 0
in_progress = 0
needs_review = 0
review result = failed
blocking finding = WU003 latest checkpoint acceptanceCriteriaResult is partial

M9 must make this state manageable without mutating historical checkpoints or manually editing canonical state.

2. Objective

M9 adds the missing workflow-management and terminal-review layer on top of the validated workflow engine.

M9 solves four problems:

1. aiqt review can get stuck forever on a known historical finding.
 2. AIQT lacks a manager report that explains project health and next action.
 3. AIQT cannot clearly distinguish development-complete from production-ready.
 4. aiqt export all does not produce a final release-readiness report.
-

3. In Scope

M9 includes:

- aiqt manage and aiqt manage --json
 - aiqt review --mode development | release
 - aiqt review acknowledge <finding-key> --reason "..."
 - aiqt next --preview
 - aiqt next cancel
 - aiqt export all adds final-review.md
 - stale NEXT-NOT-IMPLEMENTED warning cleanup
 - lightweight graph and plan-quality warnings
-

4. Out of Scope

M9 must not include:

- aiqt checkpoint amend
- aiqt dependency update
- aiqt graph amend

- aiqt issue promote
- aiqt repair plan
- integration skills bootstrap
- package publishing
- direct AI/API execution
- full requirement lifecycle management
- full packet body persistence
- manual rewrite of existing checkpoint history

These remain candidates for later milestones.

5. Repository Implementation Scope

Likely implementation areas:

```
src/cli/
src/commands/review*
src/commands/manage*
src/commands/next*
src/commands/export*
src/core/review/
src/core/manage/
src/core/workflow/
src/core/state/
src/core/runlog/
src/core/export/
src/schemas/
tests/review*
tests/manage*
tests/next*
tests/export*
tests/fixtures/
```

Implementation may differ if the repository already has better module boundaries. Keep changes localized and consistent with existing M1-M8 patterns.

6. Runtime and Package Impact

No new runtime package dependency is expected.

M9 must not add:

- external API calls

- AI-provider SDKs
- connector/plugin dependencies
- cloud-service dependencies
- package-publishing logic

All commands must work offline against local .aiqt/ state files.

If the implementation introduces a dependency, the reason must be justified in the implementation report.

7. Schema Contract

7.1 Schema version decision

M9 adds an optional review acknowledgment structure. This is an additive, backward-compatible state extension.

Decision: do not bump AIQT_SCHEMA_VERSION in M9.

Rules:

- Existing state files without review must remain valid.
- Missing review.acknowledgedFindings must be treated as an empty array.
- No existing field may change meaning.
- No historical checkpoint may be rewritten.

7.2 New state field

Add optional state shape equivalent to:

```
{
  "review": {
    "acknowledgedFindings": [
      {
        "findingKey": "checkpoint:WU003:acceptanceCriteriaResult:partial",
        "reason": "Live Clerk verification requires user-owned setup and is accepted as a
development-complete limitation.",
        "acknowledgedAt": "2026-07-14T00:00:00.000Z",
        "sourceCommand": "aiqt review acknowledge"
      }
    ]
  }
}
```

Recommended field rules:

- findingKey: required non-empty string.
- reason: required non-empty trimmed string.
- acknowledgedAt: required ISO timestamp.
- sourceCommand: required string, usually aigt review acknowledge.

Optional future fields such as acknowledgedBy are out of scope unless already supported by project patterns.

7.3 Finding Key Contract

M9 must introduce stable finding keys for acknowledgment. Finding keys are durable identities for review findings. They are separate from human display IDs such as FIND-001.

Rules:

- Finding keys must be deterministic and stable across review runs for the same underlying condition.
- Finding keys must be derived from canonical state fields rather than display order.
- Finding keys must not use unstable display IDs such as FIND-001.
- Finding keys must not include timestamps unless the finding itself is timestamp-specific.
- Finding keys must use lowercase category prefixes and colon separators.
- Display IDs may still be used in human output, but acknowledgment must use findingKey.

Required key pattern for the dogfood regression:

checkpoint:<workUnitId>:<field>:<value>

Required key for WU003:

checkpoint:WU003:acceptanceCriteriaResult:partial

General key patterns:

checkpoint:<workUnitId>:<field>:<value>

state:<entityType>:<entityId>:<rule>

dependency:<dependencyId>:<rule>

workunit:<workUnitId>:<rule>

milestone:<milestoneId>:<rule>

export:<exportName>:<rule>

review:<rule>

Examples:

checkpoint:WU003:acceptanceCriteriaResult:partial

dependency:DEP-031:late-stage-relates-to

workunit:WU025:empty-scope

workunit:WU023:suspicious-late-stage-ready
export:project-plan:requirements-status-not-tracked
state:lastAgentPacket:stale-reference

If a new review rule does not fit these examples, the implementation must add a documented key pattern with tests.

8. Command Contract

8.1 aiqt manage

Add:

aiqt manage
aiqt manage --json

Purpose: produce a read-only project manager report.

Required human-readable sections:

- Project status
- Current workflow position
- Work unit counts
- Milestone completion summary
- Ready, blocked, and needs-review work
- Active review findings
- Acknowledged findings
- Open checkpoint issues
- User-action-required items
- External verification gaps
- Agent-fixable unresolved issues
- Development-complete classification
- Production-ready classification
- Recommended next command
- Reason for recommendation

Required JSON fields should be deterministic and include at least:

```
{  
  "projectStatus": "review",  
  "developmentComplete": true,  
  "productionReady": false,  
  "recommendedCommand": "aiqt review --mode release",  
  "reason": "All work units are done, but production-readiness blockers remain.",  
  "counts": {
```

```
"ready": 0,  
"planned": 0,  
"in_progress": 0,  
"needs_review": 0,  
"done": 26  
,  
"activeFindings": [],  
"acknowledgedFindings": [],  
"userActionRequired": [],  
"externalVerificationGaps": [],  
"agentFixableIssues": []  
}
```

The exact schema can follow existing CommandResult conventions.

Final-state behavior

When all work units are done, aigt manage must explicitly distinguish:

Development complete: yes

Production ready: no

Production can be false because of user setup, live-service verification, legal/compliance review, or unresolved agent-fixable issues.

8.2 aigt review --mode

Add:

```
aiqt review --mode development
```

```
aiqt review --mode release
```

Default mode

Decision: default aigt review mode is development.

Therefore:

```
aiqt review
```

is equivalent to:

```
aiqt review --mode development
```

Development mode semantics

Development mode answers: “Can the AI-assisted development workflow continue or be considered development-complete?”

Rules:

- Exit 0 if no unacknowledged blocking review findings exist in development mode.
- Acknowledged findings remain visible but do not fail the command.
- User-action-required and production-readiness issues do not fail development review unless they are represented by unacknowledged blocking review findings.
- Historical external-verification findings can be acknowledged.
- M9 does not add a developmentBlocking field to ReviewFinding; development-mode blocking behavior is derived from the existing blocking flag plus acknowledgment state.

Release mode semantics

Release mode answers: “Is this project production-ready?”

Rules:

- Exit 1 if any release blocker exists.
- Acknowledged findings remain visible and may still block release.
- productionReady = false means exit 1.
- External-service setup gaps, legal review gaps, live smoke-test gaps, and unresolved security concerns should be release concerns.

Output

Both modes must show:

- mode
 - active findings
 - acknowledged findings
 - developmentComplete
 - productionReady
 - exit-relevant reason
 - nextRecommendedCommand
-

8.3 aiqt review acknowledge

Add:

```
aiqt review acknowledge <finding-key> --reason "<reason>"
```

```
aiqt review acknowledge <finding-key> --reason "<reason>" --json
```

Purpose: acknowledge a known finding without rewriting the original checkpoint or deleting the finding from history.

Rules:

- finding-key must match an active finding key generated by current review rules.

- --reason is required and must be non-empty after trim.
- Duplicate acknowledgment of the same finding key should be idempotent or update only if existing project conventions support update semantics. Prefer idempotent no-op with success.
- Append a canonical runlog event.
- Recompute nextRecommendedCommand if needed.

Expected dogfood use:

aiqt review acknowledge checkpoint:WU003:acceptanceCriteriaResult:partial --reason "Live Clerk verification requires user-owned setup and is accepted as a development-complete limitation."

After acknowledgment:

aiqt review --mode development

should pass if no other unacknowledged development blockers exist.

aiqt review --mode release

should still surface production-readiness concerns.

8.4 aiqt next --preview

Add:

aiqt next --preview

aiqt next --preview --json

Purpose: show what aiqt next would select without mutating state.

Required output:

- selected work unit
- selected milestone
- readiness reason
- satisfied blocking dependencies
- sequencing warnings
- whether packet generation would be allowed
- mutation: false

Must not:

- modify any work unit status
- set currentWorkUnitId
- set currentMilestoneId

- create or update lastAgentPacket
- append packet-created events

Do not append runlog events for preview. It is read-only.

8.5 aiqt next cancel

Add:

aiqt next cancel

aiqt next cancel --json

Purpose: undo an accidental packet selection before any checkpoint exists for that packet.

Allowed only when:

- currentWorkUnitId is set.
- Current work unit status is in_progress.
- lastAgentPacket.workUnitId equals currentWorkUnitId.
- No checkpoint exists for that work unit after the current packet was created.

Mutation:

- current work unit: in_progress -> ready or planned based on dependency readiness
- currentWorkUnitId -> null
- currentMilestoneId -> recomputed
- lastAgentPacket -> previous valid packet or null
- nextRecommendedCommand -> recomputed
- append packet.cancelled event to runlog

Do not delete runlog history. Do not remove the original packet-created event.

8.6 aiqt export all final-review output

Extend:

aiqt export all

to generate:

.aiqt/exports/final-review.md

Required sections:

- Executive summary
- Development completion status

- Production readiness status
- Work graph completion
- Milestone completion
- Validation summary
- Blocking findings
- Acknowledged findings
- Open checkpoint issues
- User-action-required checklist
- External verification gaps
- Agent-fixable unresolved issues
- Release blockers
- Post-MVP backlog candidates
- Recommended next actions

The report must not claim production readiness when unresolved setup, legal, live-smoke, or release blockers exist.

9. Exit Code Contract

Use existing exit-code conventions.

Code	Meaning
0	Success
1	Quality or review failure
2	Blocked workflow condition
3	Invalid input or invalid/corrupt state
10	No input

aiqt manage

Scenario	Exit code
Report generated	0
Missing .aiqt/ or invalid state	3

aiqt review --mode development

Scenario	Exit code
No unacknowledged development blockers	0
Unacknowledged development blocker exists	1
Invalid mode or invalid state	3

aiqt review --mode release

Scenario	Exit code
Production ready	0
Production not ready / release blocker exists	1
Invalid mode or invalid state	3

aiqt review acknowledge

Scenario	Exit code
Acknowledgment stored or already present	0
Finding key not found in current active findings	3
Missing or empty --reason	3
Invalid state	3

aiqt next --preview

Scenario	Exit code
Preview generated	0
No ready work unit / project in review	2
Invalid state	3

aiqt next cancel

Scenario	Exit code
Packet cancelled	0
No current packet to cancel	2
Checkpoint already exists for current packet/work unit	2
Invalid or inconsistent state	3

10. Runlog Event Contract

M9 must use the established canonical runlog event shape.

Canonical shape:

```
{  
  id: string;  
  type: string;  
  timestamp: string;  
  actor: string;
```

```

summary: string;
relatedIds: string[];
data: Record<string, unknown>;
}

```

Add mutation events only. Do not append runlog events for read-only commands such as aigt manage or aigt next --preview.

10.1 review.finding_acknowledged

Required event shape:

```

{
  "id": "EVT-...",
  "type": "review.finding_acknowledged",
  "timestamp": "2026-07-14T00:00:00.000Z",
  "actor": "cli",
  "summary": "Acknowledged review finding
checkpoint:WU003:acceptanceCriteriaResult:partial",
  "relatedIds": ["WU003"],
  "data": {
    "findingKey": "checkpoint:WU003:acceptanceCriteriaResult:partial",
    "reason": "Live Clerk verification requires user-owned setup and is accepted as a
development-complete limitation.",
    "sourceCommand": "aiqt review acknowledge"
  }
}

```

10.2 packet.cancelled

Required event shape:

```

{
  "id": "EVT-...",
  "type": "packet.cancelled",
  "timestamp": "2026-07-14T00:00:00.000Z",
  "actor": "cli",
  "summary": "Cancelled agent packet PKT-004 for work unit WU023",
  "relatedIds": ["PKT-004", "WU023", "M008"],
  "data": {
    "packetId": "PKT-004",
    "workUnitId": "WU023",
    "milestoneId": "M008",
    "restoredWorkUnitStatus": "planned",
    "previousLastAgentPacketId": "PKT-003",
  }
}

```

```
"sourceCommand": "aiqt next cancel"  
}  
}
```

If an existing runlog helper generates id, timestamp, or actor, the implementation must use that helper rather than duplicating event construction.

11. Review and Manage Warning Rules

M9 adds lightweight warnings only.

11.1 Suspicious late-stage readiness

Warn when late-stage work is ready while earlier core work remains incomplete.

Keyword candidates:

i18n
accessibility
release
hardening
readiness
polish
content pass
production

Warning should be non-blocking.

11.2 Mixed inbound dependency types

Warn when a work unit has both inbound blocks and relates_to dependencies.

This is the WU025 pattern and belongs to the same family as the original WU023 incident.

Warning should recommend checking whether the relates_to edge is actually a prerequisite.

11.3 Potential orphaned requirement fragment

Warn when a work unit references a requirement but the work unit scope appears to cover only part of that requirement and no other work unit obviously covers the remaining portion.

M9 should keep this heuristic and non-blocking.

Do not build a full semantic requirement-coverage engine in M9.

12. Export Quality Fixes

12.1 Requirement draft labels

Problem: project-plan.md can show all requirements as (draft) even when all work units are done.

M9 decision:

- Remove requirement status labels from project-plan.md exports.
- Do not replace them with a pseudo-status.
- Do not implement full requirement lifecycle in M9.

Rationale: AIQT does not currently manage requirement lifecycle state. Exporting (draft) after completion is misleading; inventing status not tracked adds noise without improving state fidelity.

12.2 Agent packet export

Problem: packet export contains only packet metadata when packet text is not stored.

M9 decision:

- Keep the honest metadata-only behavior.
- Do not invent packet text.
- Add clearer wording that full packet text is unavailable because it is not stored in canonical state.

Full packet persistence is out of scope.

12.3 Stale status warning

Remove or replace:

NEXT-NOT-IMPLEMENTED

Full aiqt next behavior (agent packet generation) is not implemented yet.

Replacement:

All work units are done. Project is in review state. Run aiqt manage or aiqt export all.

13. Test Matrix

13.1 Unit tests

- Deterministic finding key generation.
- Finding acknowledgment lookup.
- Missing review object treated as empty acknowledgment list.
- Development review mode ignores acknowledged findings for exit-code purposes.

- Release review mode still surfaces acknowledged findings.
- Manager classification: development complete vs production ready.
- Preview selection does not mutate state.
- Cancel eligibility logic.
- Late-stage keyword warning detection.
- Mixed inbound dependency warning detection.

13.2 Integration tests

- aigt review acknowledge writes state and appends canonical runlog event.
- aigt review --mode development passes after acknowledgment.
- aigt review --mode release still fails when production blockers exist.
- aigt next --preview produces same target as aigt next would, without mutation.
- aigt next cancel returns work unit to computed ready/planned state and clears current pointers.
- aigt export all creates final-review.md.

13.3 Regression tests

Create or adapt a fixture representing the completed Natural Medicine dogfood state:

projectStatus = review

26 work units done

0 ready/planned/in_progress/needs_review

latest WU003 checkpoint acceptanceCriteriaResult = partial

Required assertions:

- Review fails before acknowledgment.
- Acknowledge deterministic finding key succeeds.
- Development review passes after acknowledgment.
- Release review still reports release concerns.
- aigt manage reports developmentComplete = true and productionReady = false.
- final-review.md includes acknowledged finding and release-readiness blockers.

13.4 CLI contract tests

- Invalid review mode returns exit 3.
- Missing acknowledge reason returns exit 3.
- Unknown finding key returns exit 3.
- next --preview in review state returns exit 2.
- next cancel with no current packet returns exit 2.
- next cancel after checkpoint exists returns exit 2.

13.5 Export tests

- final-review.md is generated by export all.

- final-review.md includes development completion status.
 - final-review.md includes production readiness status.
 - project-plan.md no longer misleadingly marks all requirements as draft after completion.
 - Stale packet export wording is clarified.
-

14. Definition of Done

M9 is complete when:

1. aiqt manage works in human mode.
 2. aiqt manage --json is deterministic and test-covered.
 3. aiqt review acknowledge stores an auditable acknowledgment.
 4. Acknowledged findings remain visible.
 5. aiqt review --mode development passes when only acknowledged development blockers remain.
 6. aiqt review --mode release still surfaces production-readiness concerns.
 7. aiqt next --preview shows the next work unit without mutating state.
 8. aiqt next cancel safely cancels an uncheckpointed packet.
 9. aiqt export all generates final-review.md.
 10. final-review.md includes release-readiness classification.
 11. NEXT-NOT-IMPLEMENTED is removed or replaced.
 12. Requirement draft labels are removed from exports.
 13. Lightweight warning rules are implemented and non-blocking.
 14. Existing tests still pass.
 15. New tests cover the completed dogfood regression state.
-

15. Implementation Branch

Use:

`git checkout main`

`git pull`

`git checkout -b feat/m9-workflow-manager-review-acknowledgment`

16. Implementation Report Requirements

The implementation report must include:

- files changed
- schema changes

- commands added
- exit code behavior
- runlog events added
- tests added
- validation results
- any deviations from this spec
- any deferred items